

# AE646 — Interim Report (Stage 2)

Operator Learning for Parametric Darcy Flow: FNO vs. MLP Baseline

Team PINNacles

Aryamann Srivastava

Department of Aerospace Engineering, IIT Kanpur

September 2026

## 1 Introduction and Motivation

Traditional numerical solvers such as Finite Difference Methods (FDM) and Finite Element Methods (FEM) require re-solving the full PDE for every new set of parameters, imposing prohibitive computational cost when thousands of queries are needed (e.g., uncertainty quantification, inverse design, real-time control). Neural operators learn a mapping between infinite-dimensional function spaces and amortise this cost over many queries.

This project evaluates two architectures for the 2D parametric Darcy flow problem:

- **MLP baseline** — a fully-connected network with no structural inductive bias.
- **Fourier Neural Operator (FNO)** [1] — a resolution-invariant architecture whose trainable weights reside in the frequency domain.

Both are trained to map the permeability field  $\kappa(x, y)$  and spatial coordinates  $(x, y)$  to the pressure field  $u(x, y)$  satisfying the 2D Darcy equation:

$$-\nabla \cdot (\kappa(x, y) \nabla u(x, y)) = f(x, y), \quad (x, y) \in [0, 1]^2, \quad (1)$$

with homogeneous Dirichlet boundary conditions. The forcing  $f = 1$  is constant; the permeability  $\kappa \in \{0.1, 1.0\}$  is piecewise-constant, randomly distributed on a  $64 \times 64$  grid.

## 2 Dataset Setup

### 2.1 Source

We use the PDEBench dataset [2] (DOI: 10.18419/darus-2986), a benchmark suite for scientific machine learning. The 2D Darcy flow split provides 10 000 samples at  $128 \times 128$  resolution.

### 2.2 Preprocessing

Each sample consists of:

- **Input:**  $\kappa(x, y)$  subsampled from  $128 \times 128$  to  $64 \times 64$  by taking every second grid point (literature-standard subsampling; PDEBench’s own loaders do the same).
- **Grid coordinates:**  $(x, y) \in [0, 1]^2$  concatenated channel-wise to give a 3-channel input tensor of shape  $64 \times 64 \times 3$ .
- **Target:** pressure  $u(x, y)$  at  $64 \times 64$ .

From the 10 000-sample archive we draw a reproducible (seed 42), non-overlapping subset: a 1 000-sample train/val pool — split 900 train / 100 validation — and 200 further samples held out entirely throughout training and used only for final evaluation. These counts match those used in the original FNO work (Li et al. 2021), so our numbers are directly comparable to the literature. Data are loaded in mini-batches of 16 and normalised with training-set statistics only.

### 2.3 Why Only a 1 000-Sample Training Pool?

The original FNO Darcy experiments use  $\sim 1\,000$  training samples; matching this keeps the comparison with published numbers clean and the training cost modest. 100 validation samples give a stable loss estimate for checkpoint selection, and the 200-sample test set is large enough for generalisation statistics with standard error  $\lesssim 0.003$ .

## 3 Model Architectures

### 3.1 MLP Baseline

The MLP baseline flattens the  $64 \times 64 \times 3$  input into a vector of 12 288 features and applies three hidden layers of width 2048 with GELU activations:

$$\text{MLP} : \mathbb{R}^{12288} \xrightarrow{2048} \xrightarrow{2048} \xrightarrow{2048} \mathbb{R}^{4096} \rightarrow \mathbb{R}^{64 \times 64},$$

with a final linear projection. Total parameters:  $\approx 42$  M. The MLP is fixed to the training resolution ( $64 \times 64$ ) and cannot generalise to other grid sizes.

### 3.2 Fourier Neural Operator (FNO)

The FNO [1] processes the input as a  $64 \times 64 \times 3$  field tensor through five stages:

1. **Lifting**: a pointwise linear layer maps 3 input channels to 64 hidden channels.
2. **Four Spectral Convolution Blocks**: each block computes (i) a 2D FFT of the 64-channel field, (ii) retains only the lowest 12 Fourier modes in each direction ( $12 \times 12$  complex-valued coefficients), (iii) multiplies by learned complex weights of shape  $64 \times 64 \times 12 \times 12$ , (iv) applies an inverse FFT, and (v) adds a pointwise local convolution bypass (equivalent to a  $1 \times 1$  convolution).
3. **Projection**: two linear layers ( $64 \rightarrow 128 \rightarrow 1$ ) map hidden channels to the scalar output.

Because the learned weights operate in frequency space (not on a fixed grid), the same trained network can be evaluated at any grid resolution — the key property exploited in our zero-shot super-resolution experiments (Section 6). Total parameters (baseline config):  $\approx 4.7$  M.

## 4 Training Protocol

Both models are trained with:

- **Loss**: Relative  $L_2$  error  $\|\hat{u} - u\|_2 / \|u\|_2$  per sample, averaged over the batch.
- **Optimiser**: Adam, learning rate  $10^{-3}$ , cosine annealing schedule (min LR  $10^{-5}$ ).
- **Batch size**: 16; up to 100 epochs (200 for the larger FNO-improved configuration). No early stopping — every run completes its full epoch budget and the checkpoint with the lowest validation relative- $L_2$  error is retained.
- **Hardware**: NVIDIA RTX PRO 6000 (CUDA), PyTorch 2.x.

#### 4.1 FNO Improved Configuration

An extended FNO configuration (`fno_improved`) was also trained with doubled width (128 channels), more Fourier modes (20 per direction, up from 12), and 6 spectral blocks (up from 4), totalling  $\approx 78.8$  M parameters. This represents a scaling experiment to probe the accuracy–parameter trade-off.

### 5 Preliminary Quantitative Results

All metrics are computed on the held-out 200-sample test set. Error is relative  $L_2$ :  $\epsilon_{\text{rel}} = \|\hat{u} - u\|_2 / \|u\|_2$ .

Table 1: Test-set performance at training resolution ( $64 \times 64$ ). Lower is better.

| Model        | Params | Mean          | Median        | Std    | Min           | Max    |
|--------------|--------|---------------|---------------|--------|---------------|--------|
| MLP baseline | 42.0 M | 0.0820        | 0.0695        | 0.0456 | 0.0325        | 0.3367 |
| FNO original | 4.7 M  | 0.0521        | 0.0398        | 0.0451 | 0.0155        | 0.3467 |
| FNO improved | 78.8 M | <b>0.0456</b> | <b>0.0307</b> | 0.0487 | <b>0.0106</b> | 0.3535 |

Key observations:

1. The FNO original achieves 37% lower mean error than the MLP while using  $9\times$  fewer parameters, demonstrating the advantage of spectral inductive biases for smooth PDE solutions.
2. FNO improved reduces mean error a further 12% over FNO original, at the cost of a  $17\times$  parameter increase — indicating diminishing returns in the scaling regime tested.
3. All models exhibit right-skewed distributions: most samples have errors well below 0.1, but a small fraction ( $\lesssim 5\%$ ) with highly heterogeneous  $\kappa$  fields drive errors toward 0.3 (see Figure 1).

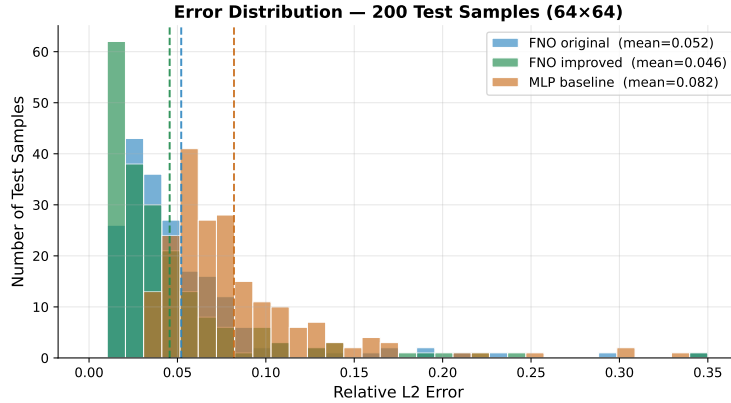


Figure 1: Error distribution across 200 test samples. Dashed vertical lines indicate per-model means. FNO errors are concentrated at lower values; MLP’s distribution is shifted right.

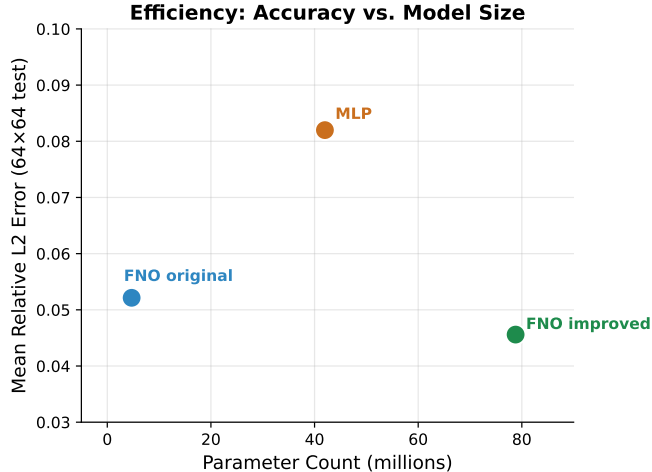


Figure 2: Accuracy–efficiency scatter: FNO original occupies the favourable lower-left region (high accuracy, low parameter count). Scaling FNO to 78.8M params yields only marginal accuracy gain.

## 6 Zero-Shot Super-Resolution

Because FNO’s spectral weights are resolution-agnostic, the same trained model can be queried on a finer grid without retraining. We evaluated both FNO variants at  $128 \times 128$  (the native PDEBench resolution), using the native-resolution  $\kappa$  fields set aside during preprocessing (data/processed/test\_hires.npz — genuine PDEBench ground truth, never downsampled) as input.

Table 2: Zero-shot super-resolution: mean relative  $L_2$  error at  $128 \times 128$ .

| Model        | $64 \times 64$ (trained) | $128 \times 128$ (zero-shot) |
|--------------|--------------------------|------------------------------|
| MLP baseline | 0.0820                   | —                            |
| FNO original | 0.0521                   | 0.0594                       |
| FNO improved | 0.0456                   | 0.0563                       |

The MLP cannot be evaluated at  $128 \times 128$  as its input layer is fixed at  $64 \times 64 \times 3 = 12\,288$  neurons. FNO’s error increases moderately (+14% for the original, +23% for the improved variant) at  $2\times$  resolution, which is expected since the models were trained with only 12 and 20 retained modes respectively — a limitation relative to the richer spatial structure available at  $128 \times 128$ .

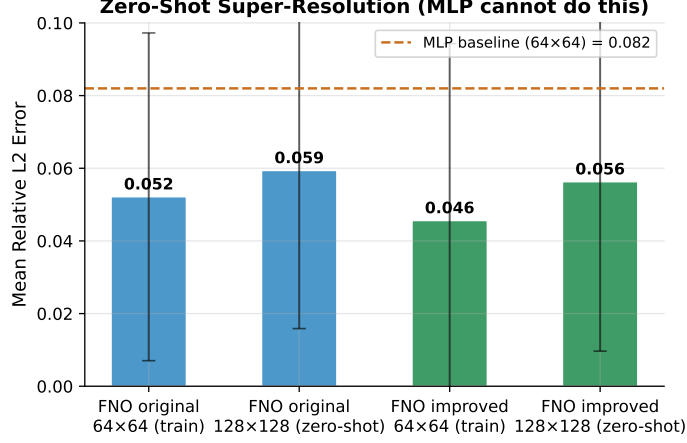


Figure 3: Zero-shot super-resolution performance. Error bars show one standard deviation. The dashed line is the MLP baseline at its maximum resolution ( $64 \times 64$ ).

## 7 Inference Speed

We benchmarked inference speed on the training hardware (NVIDIA RTX PRO 6000) with batch size 1, reporting the median of 50 timed forward passes after 5 warm-up iterations. The FDM reference is the median of 20 SciPy sparse direct solves on CPU:

Table 3: Inference speed per sample. FDM measured on CPU.

| Model / Solver  | Time (ms/sample) | Speedup vs. FDM |
|-----------------|------------------|-----------------|
| FDM (reference) | 1030             | 1×              |
| MLP baseline    | 0.13             | 7740×           |
| FNO original    | 0.71             | 1443×           |

Both neural models are several orders of magnitude faster than the FDM reference. The MLP’s lower inference latency is expected: it performs a single matrix–vector chain, whereas FNO computes 4 FFT pairs. Despite this, FNO’s superior accuracy makes it the more practical operator surrogate.

## 8 Code Quality and Reproducibility

The codebase is organised into four directories:

- `src/` — `models.py` (MLPBaseline, SpectralConv2d, FNO2d), `train.py` (training loop; keeps the best-validation checkpoint), `evaluate.py`, `superres_eval.py`, `benchmark_speed.py`, `benchmark_components.py`, `generate_figures.py`.
- `configs/` — `fno.yaml`, `fno_improved.yaml`, `mlp.yaml`.
- `results/` — one `run_00x/` per model with its `eval_metrics.json` (best epochs 92 / 99 / 198), plus `benchmark_*.json` and `figures/`.
- `docs/` — proposal, interim, and final reports.

A `pytest` suite covers model shapes, the relative- $L_2$  metric, and data loading; all results are committed. Training and evaluation reproduce with:

```
python3 src/train.py --config configs/fno.yaml
```

```
python3 src/evaluate.py --config configs/fno.yaml \
    --checkpoint results/run_001/best_model.pt
python3 src/generate_figures.py
```

## 9 Issues Encountered and Next Steps

### 9.1 Issues During Stage 1–2

1. **GPU async timing:** Initial component profiling used `time.perf_counter()` inside the GPU forward pass, measuring dispatch time rather than execution time. Fixed by calling a device-synchronisation primitive (`torch.cuda.synchronize()` on CUDA, `torch.mps.synchronize()` on Apple Silicon) before every timer read (implemented in `benchmark_components.py`). The full layer-wise breakdown is deferred to the Stage 3 report.
2. **Resolution mismatch at  $128 \times 128$ :** FNO retains only 12 Fourier modes; querying at  $128 \times 128$  means every Fourier mode above the 12th is discarded, contributing to the  $\sim 14\%$  error increase in zero-shot evaluation.
3. **MLP overfitting:** Despite GELU activations and weight decay, the MLP training curve showed larger train/val gap than FNO, suggesting the fixed-resolution architecture is less sample-efficient for this task.

### 9.2 Next Steps (Stage 3)

1. **Physical interpretation:** Visualise prediction errors on specific samples; relate high-error cases to  $\kappa$  field complexity.
2. **Extended ablation:** Vary FNO modes (8, 12, 16, 20) and width (32, 64, 128) to map the accuracy–cost Pareto frontier.
3. **Training resolution generalisation:** Train FNO directly at  $128 \times 128$  and compare zero-shot vs. native  $128 \times 128$  performance.
4. **Error correlation:** Study whether per-sample error correlates with  $\kappa$  heterogeneity (e.g., number of distinct regions, interface perimeter).

## References

- [1] Z. Li et al., “Fourier Neural Operator for Parametric Partial Differential Equations,” *ICLR*, 2021.
- [2] M. Takamoto et al., “PDEBench: An Extensive Benchmark for Scientific Machine Learning,” *NeurIPS*, 2022. DOI: 10.18419/darus-2986.